

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Progettare il prompt

Tag e struttura

Perché i tag

Il prompt B del modulo, con le istruzioni da una parte e i dati dall'altra

Il prompt B del modulo: dal messaggio di un cliente, la Falegnameria Fittizia, una **scheda** per il responsabile del turno. Qui, in breve, le sue cinque righe stanno in `<istruzioni>` e il messaggio in `<messaggio>`

`il prompt B, in breve: […] segna un taglio`

```
<istruzioni>
Sei un operatore di secondo livello dell'assistenza clienti […]
Contesto: la scheda serve al responsabile del turno […]
Istruzione: estrai dal messaggio qui sotto i fatti rilevanti […]
Formato: una tabella con le colonne Problema | Quando | […]
Vincoli: […] ometti nome e numero di telefono del cliente.
</istruzioni>
<messaggio>
Buongiorno, vi scrivo per la Falegnameria Fittizia di Rovereto.
Venerdì mattina il centralino non funzionava […]
</messaggio>
```

I tag XML aiutano Claude a leggere **senza ambiguità** un prompt che mescola istruzioni, contesto, esempi e dati — soprattutto quando i dati sono tanti

Un tag per ogni tipo di contenuto (per esempio `<instructions>`, `<context>`, `<input>`): meno interpretazioni sbagliate — e il testo del cliente resta **un dato**, non un'istruzione

Regole dei tag

Nomi coerenti, tag annidati, e ogni testo al suo posto

- Nomi **descrittivi**, e **sempre gli stessi** in tutti i vostri prompt: se il messaggio del cliente sta in `<messaggio>`, ci sta in ogni prompt
- Tag **annidati** quando il contenuto ha una gerarchia: più documenti dentro `<documents>`, ciascuno nel suo `<document>`, con il testo in `<document_content>` e la provenienza in `<source>`
- Gli **esempi** in `<example>`, e più esempi in `<examples>`: così Claude li distingue dalle istruzioni — li riprende la parte «Esempi: quando aiutano, quando danneggiano»
- Il testo **incollato da altrove**, marcato come tale: nella sua guida per Opus, Anthropic suggerisce di distinguere il testo vostro da quello incollato, con un tag `<pasted_content>` — lo riprende il Modulo 7
- Anthropic per prima usa i tag: il system prompt di claude.ai che pubblica, quello di Sonnet, è diviso in sezioni come `<claude_behavior>` e `<product_information>`

Tag anche nella risposta

La struttura si chiede, e i modelli recenti la seguono

- Chiedete che certe **sezioni della risposta** arrivino dentro tag dedicati: «scrivi la scheda dentro `<scheda>` e i dati mancanti dentro `<mancanti>`» — poi un programma ritaglia ogni sezione
- **La struttura si chiede**: i modelli recenti seguono anche strutture complesse, quando glielo si dice; e per togliere i preamboli («Ecco la scheda…»), un'istruzione diretta nel system prompt
- Il **prefill** — scrivere voi l'inizio della risposta, che Claude continua — nell'API, sui modelli recenti, **non è più supportato**: la richiesta torna con un errore
- Per separare le sezioni, tag XML o titoli Markdown: la forma esatta, osserva Anthropic, conta probabilmente **sempre meno**

Solo JSON

M2-D03, 1 di 2: «SOLO un JSON», chiesto a parole, sui tre modelli

Indovinate, a voce: chiedo «rispondi SOLO con un JSON valido» — arriverà **pulito**? E il **telefono** del cliente ci sarà dentro?

`richiesta-json.md, con il messaggio accorciato: […]`

```
Estrai dal messaggio i campi seguenti e rispondi SOLO con un JSON
valido, senza testo prima o dopo, con esattamente queste chiavi:
tipo_cliente (privato, azienda o associazione), servizio,
problema, urgenza (bassa, media, alta o urgente),
azione_richiesta.

<messaggio>
Buongiorno, vi scrivo per la Falegnameria Fittizia di Rovereto.
[…] Mi potete richiamare al +39 000 000 0103, la mattina. […]
</messaggio>
```

Demo **M2-D03**, dal vivo: la richiesta a **Haiku**, **Sonnet** e **Opus**, una chiamata `claude -p` per modello, effort low, nessuno strumento — il comando è nel README dell'elemento, passo 2

Guardate: il JSON è **valido**? Arriva dentro un **blocco di codice** Markdown, fra tre apici inversi? E dove finisce il **telefono**?

Solo JSON

M2-D03, 1 di 2: «SOLO un JSON», chiesto a parole, sui tre modelli

«Solo JSON», sui tre modelli

Haiku

JSON valido: sì

blocco di codice: no

telefono: sì

`chiavi: tipo_cliente, servizio, problema, urgenza, azione_richiesta`

«azione_richiesta», la fine

… richiamare il cliente al +39 000 000 0103 la mattina.

Sonnet

JSON valido: sì

blocco di codice: no

telefono: sì

`chiavi: tipo_cliente, servizio, problema, urgenza, azione_richiesta`

«azione_richiesta», la fine

… richiamare Franco Fittizio al +39 000 000 0103 la mattina.

Opus

JSON valido: sì

blocco di codice: no

telefono: sì

`chiavi: tipo_cliente, servizio, problema, urgenza, azione_richiesta`

«azione_richiesta», la fine

… richiamare il cliente (Franco Fittizio) al +39 000 000 0103 in mattinata

Registrato il 16 ottobre 2026 · Claude Code 2.1.301 · Haiku, Sonnet, Opus 5.5 · «rispondi SOLO con un JSON valido» · claude -p, nessuno strumento · di ogni risposta, le chiavi e la fine di un campo

Haiku, Sonnet e Opus, una chiamata ciascuno a effort low: le chiavi, e la fine del campo azione_richiesta (i puntini segnano il taglio)

JSON valido, nessun blocco di codice, e il telefono del cliente in un campo

© Simone Scannapieco — figura dalla registrazione del 16 ottobre 2026

- ✔ **Pulito**: tre JSON validi, con le cinque chiavi, senza blocco di codice — qui la richiesta a parole ha retto su tutti e tre i modelli
- 📞 Ma il **telefono c'è**, in coda ad `azione_richiesta`, su tutti e tre, e Sonnet e Opus aggiungono il nome — Sonnet: «… richiamare Franco Fittizio al +39 000 000 0103 la mattina». Nessuno aveva detto che cosa fare dei dati personali

La forma garantita

M2-D03, 2 di 2: la stessa richiesta, con uno schema

- Il formato si può anche **imporre**: con `--output-format json` e `--json-schema`, `claude -p` restituisce, a lavoro finito, un JSON **validato** su uno schema — solo in modalità non interattiva

```
claude -p "$(cat richiesta-json.md)" --model haiku --effort low \
  --tools "" --system-prompt "$(cat istruzione.txt)" \
  --strict-mcp-config --output-format json \
  --json-schema "$(cat schema.json)" > risposte/schema-haiku.json
python3 leggi_json.py structured_output \
  < risposte/schema-haiku.json
```

- `schema.json`: le cinque chiavi, **tutte obbligatorie**, nessuna in più; per `tipo_cliente` e `urgenza`, l'elenco dei valori ammessi
- Guardate il campo `structured_output` della risposta, con l'oggetto validato — e il telefono?
- Per chi scrive script, e per chi li fa girare sui sistemi: l'oggetto si legge da quel campo, senza ripulire il testo

La forma garantita

M2-D03, 2 di 2: la stessa richiesta, con uno schema

`structured_output` di Haiku, registrato: «…» segna un taglio

```
{
  "tipo_cliente": "azienda",
  "servizio": "Centralino telefonico",
  "problema": "Venerdì mattina (dalle 8 alle 10:30-11 circa) …",
  "urgenza": "alta",
  "azione_richiesta":
    "… 5) Richiamare il cliente al +39 000 000 0103 in orario …"
}
```

- **La forma è garantita**: le cinque chiavi, nessuna in più, i valori ammessi — un programma lo legge senza sorprese
- **Il contenuto no**: il telefono è ancora lì. Uno schema rigoroso toglie gli errori di sintassi, **non quelli di significato** (Anthropic)
- Contro i valori inventati: se un dato può mancare, il campo si fa **opzionale** (Anthropic), o si ammette un valore come «non indicato», come nel prompt B — lo schema della demo ha tutte e cinque le chiavi obbligatorie. E nell'API le **maiuscole** dei valori elencati non sono garantite

Il punto

Tag e schema: che cosa fa ciascuno

- **I tag**: le istruzioni da una parte, i dati dall'altra — un tag per tipo di contenuto, nomi coerenti, anche nella risposta
- **Lo schema**: quando la risposta finisce in un programma, `--json-schema` ne garantisce la forma; che cosa ci finisce dentro — i dati personali, i valori che mancano — si decide nel prompt e nello schema
- Nell'esercitazione «Il prompt pessimo», il **giudice** che misura i vostri prompt risponde così: uno schema, e i suoi verdetti in `structured_output`

I tag separano le istruzioni dai dati; lo schema garantisce la forma, non il contenuto.

Subito dopo: gli esempi, quando aiutano e quando danneggiano.